

C21 — How Products Work: Technical Fluency

1. What interviewers test

"What happens when you open this app and load your feed?" "Explain to a stakeholder what an API is." "Should voice requests be processed on the device or in the cloud?" "The page is slow. How do you work with engineers to find out why?" These are not coding questions. They test whether you can talk with engineers as a credible partner, reason about feasibility, cost and risk, and translate technical constraints into user outcomes. An associate PM is not expected to build systems. They are expected to know the shape of one, to ask questions that make engineers' answers more useful, and not to propose things that cannot work.

Where it shows up. Inferred from the round structures candidates commonly report: technical-fluency questions at Google, Meta, Microsoft, Amazon, Apple, Netflix, Stripe and others, ranging from short "explain this concept" prompts to scenario questions inside product and execution rounds, with more depth for technical-leaning roles [unverified: candidate-reported]. On the job it is the daily work of reading design documents, joining reviews, asking about estimates and risks, and noticing when a request has a hidden cost.

What is scored. Whether your technical understanding is accurate, practical and tied to users Inferred. Interviewers listen for a correct picture of a request's path; the ability to explain a concept simply without being wrong; sensible questions to engineers; awareness that choices (cache, queue, on-device processing) trade latency, cost, freshness, privacy and complexity; respect for reliability and security; and the discipline to say what you do not know.

Six observable behaviors:

1. You describe what happens between a user action and the result, hop by hop, and say where time and failures concentrate.
2. You explain APIs, data models, databases and pipelines accurately in plain language.
3. You name the tradeoffs of common architecture choices (caching, queues, services, on-device versus cloud).
4. You reason about performance with percentiles and budgets, and about reliability with availability targets and dependencies.
5. You cover security and privacy basics, including what data leaves the device and why.
6. You ask engineers focused questions, and you estimate and read design documents with an eye to risk.

The failure this prevents. Six patterns end technical-fluency answers: jargon used incorrectly, an answer that ignores the user, a "just cache it" with no thought of staleness, "it should be fast" with no number, treating security as someone else's job, and bluffing past the edge of your knowledge.

2. The concept from first principles

C21.1 Why technical fluency matters, and what it is not

Technical fluency is enough understanding of how software systems work to reason about what is feasible, what it costs, where it breaks, and what questions to ask. It is not the ability to write production code. A PM who lacks it makes three kinds of mistake: proposing features whose cost is hidden (a real-time feature that needs a new data pipeline), accepting engineering answers without testing them ("it's complicated"), and missing risks that are visible to anyone who knows the shape of the system (a single point of failure, a privacy leak).

What to learn, and in what order. The request path (what happens between a tap and a result), APIs and data models, how data is stored and read, the common architecture patterns and their tradeoffs, how code reaches users, performance and reliability, security and privacy, and the platform differences between mobile and web. Each section of this chapter takes one.

Limits. Depth beyond this belongs to engineers. A senior PM at a deeply technical product needs more; the Volume 2 backlog holds that material. Do not bluff: "I'm not certain how the database handles that; I'd ask the engineer what the failure mode is" is a stronger answer than a confident wrong one.

Counterexample. Asked how to speed up a slow report, a candidate says: "Just cache it." The interviewer asks what happens when the underlying data changes. The candidate has not considered that a cache stores a copy that can go stale, so the report may show old numbers, and that the right answer depends on how fresh the data must be and how often it changes (C21.5). Fluency is not knowing the word "cache"; it is knowing what it costs.

Hidden risks. Two. *False confidence*: a half-understood term used in a meeting loses engineers' trust faster than silence. *Over-reach*: telling engineers how to build instead of what must be true (C16); fluency serves better questions, not better instructions.

C21.2 A request end to end: client, network, server, data store

Almost every product interaction follows the same path (FW-C21-01). A user acts on a **client** (a phone app or a browser). The client needs the address of the server, found through the Domain Name System (DNS), which turns a name such as `example.com` into a numeric address. The client opens a connection across the network, usually secured so others cannot read it. The request arrives at a front door (a load balancer that spreads requests among

servers, often behind a content delivery network that serves common files from a location near the user). An application server runs the logic. It reads or writes a **data store** (a database), may call other internal services or outside partners (payments, maps), builds a response, and sends it back. The client then renders what it received.

Latency. **Latency** is the time between a request and its response. The user experiences the sum of every hop: name lookup, connection setup, network travel, server processing, database work, transfer of the response, and rendering on the device. Which hop dominates varies: on a slow mobile network it is often transfer and connection setup; on a fast network with a heavy page it may be rendering; on a poorly written query it is the database.

Throughput is the amount of work a system completes per unit of time (requests per second); latency and throughput are different, and improving one does not guarantee the other.

Percentiles, not averages. The average hides the slow tail that real users feel. The 50th percentile (median) is the typical experience; the 95th or 99th percentile is what the unluckiest few percent see (C12). A page with a 1.0-second median and a 6-second 95th percentile is unpleasant for one user in twenty, and at scale that is many people. Always ask for the percentile.

What can fail at each hop. The name lookup (the name does not resolve), the connection (timeouts, poor networks), the server (overloaded, a bug), the database (slow query, lock, outage), a dependency (a partner is down), and the client (an old version, a low-end device). A PM's useful question at each hop is what happens to the user when it fails, and whether the system degrades gracefully or breaks entirely. CS-C21-02 in section 10 shows a failure in the name-lookup and routing hop that took a very large service off the internet.

Human thresholds. A response under about 0.1 seconds feels instantaneous, about 1 second keeps the user's flow of thought, and around 10 seconds loses attention (Nielsen's long-standing guidance [V: Nielsen, Usability Engineering, 1993; Miller, 1968]). These are rules of thumb, not laws; measure your own users' abandonment against load time.

Why it works. A map of the path locates problems and converts "it's slow" into "which hop?"

When not to draw it in full. For a small question (a text change) it is overkill; use it when time, failure or cost is the issue.

C21.3 APIs and data models

An **API** (application programming interface) is a defined way for one piece of software to request data or actions from another. The client calls the server's API ("give me this user's orders"); a payments partner exposes an API that your server calls ("charge this card"); a platform offers APIs that outside developers build on (C19). An API is a contract: it states what can be asked, what comes back, and what errors mean. Contracts are hard to change once others depend on them, which is why versioning and deprecation matter (C19.3).

The words you will hear. An *endpoint* is one address of the API (for example, "get order"). A request has a *method* (read, create, update, delete), parameters, and often an authentication token; a response has a status (success, client error, server error) and a body. *Rate limits* cap how many calls a client may make, to protect the service. Many web APIs

follow a common style (REST) in which resources have addresses and methods act on them; others let the client ask for exactly the fields it needs (GraphQL). You need not know the differences in detail; you need to ask what the contract is and who depends on it.

Idempotency. An operation is **idempotent** if doing it more than once has the same effect as doing it once. Reading a record is idempotent; charging a card is not, unless designed to be. Networks fail, and clients retry; if a retry of "charge \$50" arrives twice, the customer may be charged \$100. The standard defense is an *idempotency key*: the client sends a unique token with the request, and the server remembers it, so a repeat returns the first result instead of repeating the action. For any product that moves money, sends messages or creates orders, ask: what happens if this request is sent twice? (C17's payments checklist, EX-C17-06, requires idempotent sends for the same reason.)

Data models and schemas. A **schema** is the defined structure of stored data: the entities (users, orders, products), their fields, and how they relate. A data model is how the world is represented: one order has many items; a user has many orders; an item belongs to many categories. Decisions in the model are hard to reverse: if a "user" can have only one address, supporting several addresses later means a migration. Questions to ask: what are the entities, what is unique, what can be many, what must never be lost, and how will this change in a year?

Why it works. Treating the API and schema as contracts focuses attention on what is hard to change. **When not to go deep.** For a new internal prototype, a rough model is fine; for anything others will build on or that holds money or personal data, the contract deserves care.

C21.4 Databases and data pipelines; reading a SQL query

A database stores data and answers questions about it. A *relational* database keeps data in tables of rows and columns, and queries are written in SQL. Other kinds (key-value stores, document stores, search indexes) trade the flexibility of relational queries for speed or scale in particular uses. PM-level fluency is knowing what a table holds, what a query returns, and why a query is slow. This section is about reading a query; C13.6a teaches writing the queries that check a metric, on the same kind of data.

Why queries are slow, and indexes. A query that must examine every row of a table (a full scan) takes time proportional to the table's size. A **database index** is a data structure that lets the database find rows by a column's value without scanning the whole table, as an index in a book lets you find a topic without reading every page. An index speeds reads on that column and costs space and some write time. CS-C21-01 shows a missing index turning a 2.1-second query into a 12-millisecond one.

Reading a SQL query (FW-C21-04). You will see queries written by analysts and engineers, and you should be able to read them and test their claims. A query has a few parts: what to return (`SELECT`), from which table (`FROM`), how tables connect (`JOIN ... ON`), which rows to keep (`WHERE`), how to summarize (`GROUP BY` with `SUM`, `COUNT`, `AVG`), and how to order or limit the result. Ask of any query: What is one row of each table (the *grain*)? What does the join do to the row count? What is filtered out, and are `NULL` values handled? What time zone are dates in? Does the total match a number you already know?

The commonest trap is a *fan-out*: joining a table of orders to a table of order items repeats each order once per item, so summing the order amount counts it several times. Worked example 2 reads one such query.

Data pipelines. A **data pipeline** is a set of automated steps that moves data from where it is produced to where it is analyzed, typically collecting events, cleaning and transforming them, and loading them into a store for reporting. Pipelines run in *batches* (for example, nightly) or as *streams* (continuously). They matter to a PM because dashboards and experiments depend on them: a late or broken pipeline makes a metric look like it moved (C13), a definition changed in the pipeline changes the metric silently, and the "freshness" of data limits what decisions are possible (a nightly batch cannot support a real-time alert). Ask: where does this number come from, how fresh is it, and who owns the transformation?

Why it works. Knowing the grain and the join behavior catches the most common data errors. **When not to read the query yourself.** For a high-stakes number, have the analyst walk you through it and reproduce a known total.

C21.5 Architecture at PM depth: services, caching, queues

Architecture is how a system is divided into parts and how they communicate. Three patterns come up constantly (FW-C21-05).

Monolith and services. A *monolith* is a single deployable program that does everything; *services* (often called microservices when small) are separate programs, each owning part of the work, that call one another over a network. Services let teams work and release independently and scale parts separately; they add network calls, more places to fail, and the need to coordinate changes across teams. Neither is better in general: small teams often do best with a monolith, large organizations with many teams often need services. A PM's question is not "which?" but "what does this boundary cost us, and who owns each side?"

Caching. A **cache** is a stored copy of data kept close to where it is needed so that it can be served faster than recomputing or refetching it. Caches exist at many levels (the browser, a content delivery network, the application, the database). A cache improves latency and reduces load, at the cost of *staleness* (the copy may be out of date) and complexity (deciding when to refresh or remove entries). The central tradeoff: how fresh must this data be? A price shown on a product page may tolerate a few minutes of staleness; an account balance may not. The effect on speed is computed in Worked example 3: average latency depends on the *hit rate* (the share of requests served from the cache), but the slow tail comes from misses.

Queues and asynchronous work. A **message queue** holds tasks or messages until a worker processes them, so that the part of the system that receives a request need not wait for slow work. Sending a confirmation email, resizing an uploaded photo or updating a recommendation can be done asynchronously: the user gets a quick response and the work finishes shortly after. The costs: the result is not immediate (the user may need to see "processing"), failures must be retried safely (idempotency again), and ordering and duplication become design questions. Queues also absorb bursts: a spike of requests is stored and worked through, instead of overwhelming the system.

Synchronous and asynchronous; consistency. In a synchronous design the caller waits for the result; in an asynchronous one it does not. When data is copied across places (caches, replicas, services), copies can briefly disagree: *eventual consistency* means they will agree given a little time. Users notice when an action they just took is not reflected ("I saved it and it's gone"), so the product must either hide the delay or accept it deliberately.

Why it works. Naming what each pattern buys and costs lets you take part in a design discussion. **When not to add them.** Each adds moving parts; "keep it simple until measurement says otherwise" is usually right for a new product.

C21.6 Cloud, deployment pipelines, environments

Most products run on rented computing in data centers operated by cloud providers, paid for by use. This turns infrastructure from a fixed cost into a variable one, and puts a *cost per request* on every feature. A PM who knows that a new recommendation call costs a fraction of a cent per request, and that the product serves a billion requests a day, will see that a fraction of a cent is a very large number (Worked example 4). Ask engineers for the cost to run a feature at expected volume, not only the cost to build it.

Environments and deployment. Code moves through *environments*: engineers' own machines, a shared *staging* environment that mimics production for testing, and *production*, where users are. A *deployment pipeline* is the automated route that builds the code, runs tests and releases it; teams that release often use automated tests and flags (C17.2) to make each release small and reversible. What matters to a PM: how long a change takes to reach users (hours or weeks), what can be rolled back and how fast (C17.5), and what differs between staging and production (a difference is where surprises hide).

Regions and failure. Providers operate in many regions and zones; a service run in one location fails if that location fails, and running in several adds cost and complexity. How much redundancy is justified depends on what an outage costs (C21.7).

C21.7 Performance, scalability, reliability

Performance is how fast the system responds (latency) and how much it handles (throughput). **Scalability** is how well it keeps performing as load grows: *scaling up* means a bigger machine, *scaling out* means more machines sharing the work. Bottlenecks move: when the web servers are fixed, the database becomes the limit. Ask: what is the current bottleneck, what load do we expect, and what is the plan if volume doubles?

Availability. **Availability** is the share of time a service works as intended. Targets are stated in "nines": 99.9% allows about 43.8 minutes of downtime in a month; 99.99% about 4.4 minutes; 99% about 7.3 hours. Each extra nine costs far more than the last, so the right target depends on what an outage costs the users and the business. **Dependencies multiply:** a feature that needs three services in a chain, each 99.9% available, is available only $0.999^3 = 99.7\%$ of the time, about 131 minutes of downtime a month. Reliability is a product decision as well as an engineering one.

Service level objectives. A **service level objective** (SLO) is a target for a measured aspect of service quality over a period, such as "99.9% of requests succeed within 500 milliseconds, measured monthly". The measure is a *service level indicator* (SLI), a promise to customers is a *service level agreement* (SLA, with consequences if missed), and the allowed shortfall is the *error budget*. A 99.9% monthly objective gives an error budget of 43.8 minutes. If incidents have used 30 of them by mid-month, 13.8 minutes remain, and the team may slow risky launches until the budget recovers. This turns "how reliable must we be?" into a number that product and engineering can plan around (the practice is described publicly in Google's site reliability engineering materials [Public](#) [V: [Google SRE book](#)]).

Why it works. Numbers make reliability a tradeoff you can discuss. **When not to chase nines.** For an internal tool or an experiment, a lower target saves cost; spend reliability effort where an outage hurts users.

C21.8 Security and privacy basics, including on-device versus cloud processing

Security basics a PM should know. *Authentication* verifies who someone is (a password, a code, a biometric); *authorization* decides what that person may do. Data should be protected *in transit* (encrypted across the network) and *at rest* (encrypted where stored). Apply the principle of *least privilege* (give each person and system only the access needed). Keep secrets (keys, passwords) out of code. Plan for abuse as well as misuse (C22): who might try to misuse this feature, and how? Security review belongs early in design (C16.7), when changes are cheap.

Privacy basics. Collect only the data you need (*data minimization*), say why (*purpose limitation*), keep it no longer than needed, let people see and delete their data where the law gives that right, and design so that a breach exposes as little as possible. Privacy laws differ by place and change: an overview as of 2026-10 [as of 2026-10] includes the European Union's General Data Protection Regulation (in force since 2018), California's consumer privacy law and its later amendments, and India's Digital Personal Data Protection Act of 2023, with its rules notified on 13 November 2025 and the main obligations applying from 13 May 2027 [V: [law-firm summaries of the DPDP Rules](#)]. These are not legal advice; involve legal and privacy specialists early (C16.7).

On-device versus cloud processing. **On-device processing** is running computation on the user's own device instead of sending data to a server. The choice is among latency, privacy, capability, cost, battery, offline use and speed of improvement (FW-C21-07):

Factor	On-device	Cloud
Latency	No network wait; limited by the device's speed	Fast compute, plus network round trip, which varies
Privacy	Data stays on the device	Data is sent to and handled by a server
Capability	Limited by device memory and power; smaller models	Larger, more capable models and more data
Cost to the company	Little per request; shifts compute to the user's device	A cost per request that grows with usage

Factor	On-device	Cloud
Offline use	Works	Fails without a connection
Battery	Uses the device's battery	Uses the radio and less compute
Updates	Tied to app or system releases	Improved instantly on the server

A common design is *hybrid*: handle most requests on the device and send only the hard ones to the cloud, with a clear explanation to users of what leaves the device. Worked example 4 computes the latency, quality and cost of the three choices for a voice assistant.

C21.9 Mobile versus web platforms

A *web* product runs in a browser and updates the moment the server changes: every user gets the new version on their next load. A *native mobile* app is installed from a store, updates only when users update (so several versions are in use at once), and can use device features (camera, sensors, notifications) more fully. Consequences for a PM:

- *Release speed and fixes.* A web bug can be fixed in hours; a native bug may take days to ship and weeks to reach most users. Server-controlled behavior (flags, remote configuration) lets a native app change without a new release (C17.2).
- *Old versions.* A native API must keep supporting old app versions (C19.3), since some users never update.
- *Store rules.* Native apps are reviewed and governed by the store's policies (C19.4).
- *Offline and connectivity.* Native apps can store data and work offline; web apps can too, with more effort.
- *Notifications and engagement.* Push notifications are strong for re-engagement and easy to overuse (C10.9).
- *Cross-platform tools* let one codebase serve several platforms, trading some native feel and performance for cost.

Regional Lens: India. In markets where many people use lower-priced Android phones on variable mobile networks, page weight, memory use and offline behavior decide whether a product works at all Inferred [unverified: author's synthesis; no source checked]. Technology companies have published lightweight versions of apps for such conditions Public [V: Android Go apps (Google Go, Maps Go and YouTube Go, which was retired in 2022) and Facebook Lite, 2017 coverage]. The consequence for a PM: set a performance budget (for example, the app must work on a phone with limited memory and on a slow connection), test on representative devices and networks and not on the team's own phones, and treat data cost as a user cost (C09). The numbers vary widely by region and over time; check current device and network data before designing.

C21.10 Technical debt; build versus buy versus partner

C16.4 defined technical debt and showed how to discuss it as an investment with a payback; C18.7 gave the build-buy-partner chooser. The technical lens adds three questions. *For debt*: what part of the system is affected, and what happens if it fails or cannot change (a fragile billing module is a larger risk than a fragile reporting script)? *For build*: what must we maintain, secure and scale afterwards, and do we have people who can? *For buy or partner*: what is the interface (an API), how will we depend on it, what happens if the vendor changes terms, is down or goes away, and how hard is it to leave (a switching cost, C18.5)? Technical fluency helps you ask whether an integration is a one-day task (a well-documented API) or a quarter's work (a custom data migration).

C21.11 Estimating engineering effort; reading design documents

You do not estimate effort for engineers, but you can anticipate what drives it (FW-C21-08). Effort rises with: a *new data model or a migration* of existing data, *external dependencies* (another team's work, a vendor), *security or privacy review*, *scale* requirements, support for *several platforms or old versions*, *unknowns* (a technology the team has not used), and *quality bar* (accessibility, reliability). Ask engineers for a range with assumptions, and the riskiest part (C16.4).

Reading a design document. Engineers write design documents before big work. Read them for: the goal and non-goals (do they match the product goal?), the alternatives considered and why one was chosen, the data model and API (the contracts), the failure modes and what the user sees when something fails, how it will be rolled out and rolled back, the cost to run, and open questions. Ask one or two focused questions rather than many: "What happens to the user if the queue backs up?" "What is the plan if the migration fails halfway?" A PM who reads design documents carefully earns trust and catches product risks early.

3. Frameworks

Eight frameworks, set out below. The estimate conversation (FW-C16-04), the technical-debt conversation (FW-C16-05), the build-buy-partner chooser (FW-C18-07), the developer-experience funnel (FW-C19-04), the rollout ladder (FW-C17-02) and the percentile guidance in C12 are reused, not repeated.

FW-C21-01 · The request-path map · Original

- **Inputs:** a user action and the system that serves it.
- **Outputs:** the hops from client to data store and back, with for each hop what it does, what can fail, its share of the latency, and who owns it.
- **Assumptions:** the main path is known or can be learned from engineers.
- **Tradeoffs:** a simple map hides detail and finds the right hop.
- **Failure modes:** treating "the server" as one box; ignoring name lookup, network and the device; averages instead of percentiles.

- **Worked example:** Worked example 1 and CS-C21-02.

Hop	What it does	What can fail	User sees
-----	--------------	---------------	-----------

FW-C21-02 · The engineer conversation questions · Original

- **Inputs:** a proposed feature or a design document.
 - **Outputs:** ten questions that surface cost, risk and user impact.
 - **Assumptions:** engineers welcome focused questions asked early.
 - **Tradeoffs:** many questions feel like an audit; pick the two that matter most.
 - **Failure modes:** asking to be impressive; accepting "it's complicated"; telling engineers how to build.
 - **Worked example:** IQ-C21-04 and IQ-C21-07.
1. What is the riskiest part, and what would reduce the risk?
 2. What changes in the data model?
 3. What does this depend on outside our team?
 4. What happens to the user when it fails?
 5. What happens if the request is sent twice?
 6. How fresh does the data need to be?
 7. What does it cost to run at expected volume?
 8. What happens at ten times the load?
 9. How will we roll it out and roll it back?
 10. What are we not building, and what will that cost later?

FW-C21-03 · The latency budget · Original

- **Inputs:** a user-visible action and a target response time.
- **Outputs:** a budget split across hops, the measured percentiles against it, and the first hop to fix.
- **Assumptions:** timings can be measured per hop.
- **Tradeoffs:** optimizing a small hop wastes effort; the largest contributor to the tail is the one to attack.
- **Failure modes:** averages; optimizing without measuring; ignoring the slow device and network.
- **Worked example:** Worked example 1.

Steps: **Set the target at a percentile → List the hops → Measure each at that percentile → Find the largest contributor → Fix it → Re-measure.**

FW-C21-04 · The SQL read-and-reason checklist · Original

- **Inputs:** a query and its result.
- **Outputs:** answers to six checks (grain, join behavior, filters, null handling, time zone, total against a known number) and a verdict on whether to trust the result.
- **Assumptions:** you can inspect a small sample of the underlying rows.
- **Tradeoffs:** the check takes minutes and prevents wrong decisions.
- **Failure modes:** summing across a fan-out; filtering out nulls silently; mixing time zones; trusting an unchecked total.

- **Worked example:** Worked example 2.

Check	Question
Grain	What is one row of each table?
Join	Does it repeat rows (fan-out) or drop them?
Filter	What is excluded, and is that intended?
Nulls	How are missing values handled?
Time	Which time zone and what date boundary?
Total	Does it match a number I already trust?

FW-C21-05 · The architecture tradeoff table · Original

- **Inputs:** a design pattern under discussion (services, cache, queue, synchronous or asynchronous, replica).
- **Outputs:** for each pattern what it buys, what it costs, and the question a PM should ask.
- **Assumptions:** the problem it is meant to solve is stated.
- **Tradeoffs:** every pattern adds parts; simplicity is a feature.
- **Failure modes:** adopting a pattern for fashion; ignoring staleness and failure handling.
- **Worked example:** Worked example 3.

Pattern	Buys	Costs	Ask
Services	Independent teams and scaling	Network calls; coordination	Who owns each side?
Cache	Speed; lower load	Staleness; invalidation	How fresh must it be?
Queue	Responsiveness; burst absorption	Delay; retries; ordering	What does the user see while waiting?
Replicas	Read capacity; resilience	Copies disagree briefly	Can the user see stale data?

FW-C21-06 · The reliability targets sheet · Original

- **Inputs:** a service, what an outage costs and its dependencies.
- **Outputs:** an availability target in nines with the downtime it allows, the compounded availability of the dependency chain, an SLO with its error budget, and a policy for what happens when the budget is spent.
- **Assumptions:** the cost of downtime can be estimated.
- **Tradeoffs:** each nine costs more than the last.
- **Failure modes:** a target no one measures; ignoring dependency compounding; no policy for the budget.
- **Worked example:** Worked example 5.

Formulas: **downtime per month = (1 – availability) × 43,800 minutes; chain availability = product of the availabilities; error budget = allowed downtime.**

FW-C21-07 · The on-device versus cloud chooser · Original

- **Inputs:** a feature that processes user data.
- **Outputs:** a comparison on latency, privacy, capability, cost, offline use, battery and update speed, a hybrid routing rule if suitable, and what the user is told.
- **Assumptions:** the numbers for compute, network and cost can be estimated.
- **Tradeoffs:** privacy and offline use favor the device; capability and update speed favor the cloud.
- **Failure modes:** deciding on one factor; ignoring the poor-network case; no disclosure to users.
- **Worked example:** Worked example 4 and IQ-C21-03.

FW-C21-08 · Effort drivers and the design-document read · Original

- **Inputs:** a proposed project or a design document.
- **Outputs:** the effort drivers present (new data model or migration, external dependencies, security or privacy review, scale, platform and version coverage, unknowns, quality bar), the riskiest part, and a list of questions from the read.
- **Assumptions:** engineers will give a range with assumptions.
- **Tradeoffs:** reading takes time and finds risks early.
- **Failure modes:** reading only the summary; asking many questions; accepting a single date.
- **Worked example:** IQ-C21-07 and EX-C21-08.

4. Answer architecture

The technical-fluency answer, for a 20-minute round

Step	Minutes	What to produce
Clarify the scenario	2	What the feature does, for whom, what matters (speed, privacy, cost, reliability)
Map the path or components	4	The request path or the system parts; where time, data and failure concentrate
Identify the constraint	3	The binding constraint with a number (latency target, volume, availability, privacy rule)
Options and tradeoffs	6	Two or three options with what each buys and costs, using the tradeoff tables
Recommendation and risks	3	The choice, the riskiest part, the questions you would ask engineers

Step	Minutes	What to produce
Measures	2	How you would know it worked: percentile latency, error rate, availability, cost per request

Pacing rules. For an "explain X" question, give the plain definition, an everyday analogy, an example from a product, and one tradeoff, in 90 seconds, then stop. If you run behind in a scenario, keep the path map, the constraint with its number and the recommendation.

How to say it. Be plain and correct: "A cache keeps a copy near the user so it's faster; the price is that the copy can be out of date, so I'd ask how fresh this data has to be." Say what you do not know and how you would find out.

How to close. Two sentences: "I'd choose [option] because [constraint with a number], accepting [cost]. I'd ask engineering about [riskiest part], and I'd watch [percentile or error measure] to know it works."

Type: explain a concept (API, cache, database, latency)

Plain definition, analogy, product example, tradeoff, in about 90 seconds.

Type: what happens when a user does X

Walk the request path hop by hop; say where time and failures concentrate.

Type: a performance problem

Use the latency budget: percentile, hops, largest contributor, fix, re-measure.

Type: an architecture or design choice

Use the tradeoff table; ask what the choice buys and costs and who owns each side.

Type: on-device versus cloud (privacy, latency, cost)

Use the chooser, compute a number for each side, and consider a hybrid.

Type: reliability or a launch with risk

Use the reliability sheet and the launch tools from C17.

Common interviewer interventions, and what to do

Interviewer says	What it means	What to do
"What happens when I tap this?"	Request path	Walk the hops; name where it can fail
"Why is it slow?"	Diagnosis	Ask for percentiles; split by hop
"Can't we just cache it?"	Staleness	Ask how fresh the data must be
"What if the request is sent twice?"	Idempotency	Describe a key that makes repeats safe

Interviewer says	What it means	What to do
"Where does that number come from?"	Pipeline	Ask about source, freshness and transformation
"How reliable does it need to be?"	Availability	Give a target, the downtime it allows and the cost
"I don't know"	Your edge	Say what you would ask, and to whom

5. Worked examples

All five examples are **Illustrative**: the products are fictional or use a product type as the setting only, and every figure is an invented assumption. The arithmetic was computed. Each is written as notes you could say aloud.

Worked example 1 — Where does the time go? A product page's request path (Illustrative)

The situation. A shopping app's product page takes about 2.3 seconds at the median and 5.2 seconds at the 95th percentile, and the team's target is a 95th percentile under 3 seconds. The PM asks engineering for the timing of each hop (FW-C21-01 and FW-C21-03).

Median timeline (milliseconds, invented).

Hop	Median	What it is
Name lookup (DNS)	60	Finding the server's address
Connection setup and security handshake	180	Opening a secure connection
Server compute	300	Application logic and calls to other services
Database	420	The product and price queries
Transfer (600 KB over 6 Mbps)	800	Sending the page and images (600 × 8 ÷ 6,000 seconds)
Rendering and scripts on the device	500	Drawing the page
Total	2,260	

The slow tail. At the 95th percentile (invented): lookup 120, connection 300, server 500, database 2,000, transfer 1,400 (slow networks), rendering 900: total 5,220 ms. The database is the largest contributor in the tail (2,000 of 5,220), though it is only the third largest at the median. An average would have hidden this.

Fixes, in order of effect on the tail. (1) The database query scans a large table; an index on the lookup column (C21.4) cuts the tail from 2,000 to about 180 ms. (2) Images are uncompressed; compressing them halves the bytes, cutting the tail transfer from 1,400 to about 700 ms. (3) A cache for the product details (C21.5) cuts server compute in the tail

from 500 to about 300 ms. After the fixes the tail is $120 + 300 + 300 + 180 + 700 + 900 = 2,500$ ms, under the 3-second target. The median becomes $60 + 180 + 150 + 60 + 400 + 500 = 1,350$ ms.

What not to do first. Rendering at 900 ms in the tail is real but a smaller piece; start with the largest contributor.

Check for side effects. The cache can serve stale prices (set a short lifetime, and bypass the cache at checkout); the index slows writes slightly; the image compression must not harm quality. Measure the percentile again after each change (re-measure is the last step in FW-C21-03).

What this shows. Splitting the time by hop at the percentile that matters pointed to the database, a fix that an average-based look and a generic "make it faster" would have missed.

Worked example 2 — Reading a SQL query and catching a fan-out (Illustrative)

The situation. An analyst reports that revenue by country last week was \$140 for the sample below, but finance says \$100. The PM reads the query (FW-C21-04). This is read-and-reason work; no one is asked to write SQL.

The data, a tiny sample (invented).

- `orders`: order 1, user u1, \$40; order 2, user u2, \$25; order 3, user u1, \$35.
- `order_items`: order 1 has items A and B; order 2 has item C; order 3 has item D.
- `users`: u1 is in India (IN); u2 is in the United States (US).

The query.

```
SELECT u.country, SUM(o.amount) AS revenue
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
JOIN users u ON u.user_id = o.user_id
GROUP BY u.country
```

Read it with the checklist. *Grain*: one row of `orders` is an order; one row of `order_items` is an item; many items can belong to one order. *Join*: joining orders to items produces one row per item, so order 1 now appears twice. That is a fan-out. *Aggregation*: `SUM(o.amount)` then adds order 1's \$40 twice.

Compute it. After the join: IN has rows for order 1 (twice, \$40 each) and order 3 (\$35): $\$40 + \$40 + \$35 = \115 . US has order 2 (\$25). The query reports $\$115 + \$25 = \$140$. The true figures are IN $\$40 + \$35 = \$75$ and US \$25, total \$100. The query overstates revenue by 40%, which explains the discrepancy exactly.

Fix. Sum revenue at the order grain without joining items (first aggregate orders by user, or leave out the items table, which the question does not need). If items are needed, count orders with a distinct order count or aggregate items separately.

Other checks. Is each date in the same time zone as finance's? Are orders with a missing user dropped by the join (and is that intended)? Does the total now match the number finance trusts (\$100)? Matching a known total is the strongest check.

What the PM does. Say, "Revenue per order is at the order grain; the join to items repeats orders. Can we recompute without the items table and check it against finance's \$100?" This is a precise, non-accusatory question that points to the cause.

What this shows. Knowing the grain and what a join does to row counts found a 40% error from the shape of the query alone.

Worked example 3 — A cache and a latency budget for search suggestions (Illustrative)

Setting: a search-suggestion service of the kind Google runs. No fact about any real system is claimed.

The situation. As users type, the product suggests completions and must respond within about 100 milliseconds to feel instant (C21.2). The backend takes about 220 ms to compute suggestions (invented). A cache can answer in about 10 ms when it has the query.

Average latency by hit rate. Average = hit rate $\times$ 10 + (1 – hit rate) $\times$ 220.

Hit rate	Average latency
70%	$0.7 \times 10 + 0.3 \times 220 = 73$ ms
90%	31 ms
95%	20.5 ms
96%	18.4 ms

The tail. The average improves smoothly, but the percentiles do not. At a 90% hit rate, 10% of requests miss, so the 95th percentile is the slow backend: 220 ms, above the target. Only when the hit rate exceeds 95% does the 95th percentile fall to the cache's 10 ms; the 99th percentile stays at 220 ms until the hit rate passes 99%. A team that looks only at the average will declare victory at 90% (31 ms) while one request in ten feels slow.

What decides the hit rate. Search queries are very unevenly distributed: a small set of popular queries accounts for most traffic. Suppose (invented) that the most popular million queries cover 90% of requests; caching them costs a modest amount of memory. Covering the next 5% might need ten times as many entries, so there is a point of diminishing returns. The cost of memory is a number to compare with the value of the faster tail.

Freshness. Suggestions that reflect what is trending right now go stale in a cache. A lifetime of 5 minutes means a surge in a new query appears in suggestions up to 5 minutes late. The product must decide how fresh suggestions must be: for a breaking-news event, a short lifetime or a faster path for trending queries; for the long tail of ordinary queries, a long lifetime is fine.

Decision. Cache the popular queries with a lifetime of minutes, send trending queries through a short-lifetime path, set the target as a 95th-percentile latency under 100 ms (not the average), and monitor hit rate and the percentile together. Ask engineering: what is the hit rate at the planned cache size, and what happens to the user when the backend is slow? (Show cached suggestions, then update.)

What this shows. The average hid the tail, the hit rate drove the speed and the cost, and freshness was the product decision.

Worked example 4 — On-device or cloud for a voice assistant (Illustrative)

Setting: a voice assistant on a phone of the kind Apple and several other companies make. No fact about any real product is claimed.

The question. Should voice requests be processed on the device, in the cloud, or both? (FW-C21-07.)

Numbers (invented).

	On-device	Cloud
Compute time	150 ms	40 ms
Network (good connection)	none	60 ms round trip + 20 ms upload
Network (poor connection)	none	400 ms round trip + 20 ms upload
Offline	works	fails
Accuracy on easy requests (80% of traffic)	95%	96%
Accuracy on hard requests (20% of traffic)	70%	90%
Cost to the company per request	about 0	\$0.002
Data leaving the device	none	the audio or transcript

Latency. On-device: 150 ms always. Cloud on a good connection: $40 + 60 + 20 = 120$ ms. Cloud on a poor connection: $40 + 400 + 20 = 460$ ms, and nothing when offline. So the cloud is a little faster on a good connection and much slower on a poor one. On variable networks, on-device is the more predictable choice.

Quality. On-device only: $0.8 \times 95\% + 0.2 \times 70\% = 90\%$. Cloud only: $0.8 \times 96\% + 0.2 \times 90\% = 94.8\%$. The cloud's advantage is on hard requests.

Hybrid. Handle requests on the device when its confidence is high (the 80% of easy requests) and send the hard 20% to the cloud, with a routing step that takes about 10 ms. Average latency: $0.8 \times 150 + 0.2 \times (10 + 120) = 120 + 26 = 146$ ms. Quality: easy requests on the device at 95% and hard requests in the cloud at 90%: $0.8 \times 95\% + 0.2 \times 90\% = 94\%$, within one point of cloud-only (94.8%) and four points above on-device only.

Cost. At 1 billion requests a day, all-cloud costs $1,000,000,000 \times \$0.002 = \2.0 million a day; the hybrid sends 20% to the cloud: \$0.4 million a day. The on-device option moves the compute cost to users' devices (battery and memory).

Privacy. On-device keeps all data local; the hybrid sends only the hard requests, and the product should say so plainly and let users control it. For sensitive categories (health, messages) the routing rule can keep requests on the device regardless of difficulty.

Recommendation. Hybrid: on-device first, cloud for hard requests, with disclosure and a user setting, a rule that keeps sensitive categories local, and a measure of the share routed to the cloud. Ask engineering: how much memory and battery does the on-device model use, how often will the model be updated, and what happens when the cloud is slow (fall back to the on-device answer)? What would change my view: on-device quality closing the gap (move more on-device), or the cloud cost falling.

What this shows. The arithmetic made the tradeoff concrete across latency, quality, cost and privacy, and the hybrid captured most of the benefit of each.

Worked example 5 — Reliability targets for a payments feature (Illustrative)

The situation. A team is adding a payment step to its app and asks what reliability to promise. The step depends on three services in a chain: the app's own service, a fraud-check service and the payment partner, each currently about 99.9% available (invented).

What nines mean (FW-C21-06). A month has about 43,834 minutes (30.44 days). 99.9% availability allows $0.001 \times 43,834 = 43.8$ minutes of downtime; 99.99% allows 4.4 minutes; 99% allows 438 minutes (over 7 hours).

The chain. Three services in series, each 99.9%: $0.999^3 = 0.997$, so about 99.7% available, or $0.003 \times 43,834 = 131$ minutes of downtime a month, three times the budget of a single 99.9% service. Five services at 99.95% each give $0.9995^5 = 99.75\%$, about 110 minutes. The feature's availability is lower than any one component's.

Choosing a target. What does an outage cost? Suppose payments bring in \$1,500 a minute at peak and average \$600 a minute (invented): 131 minutes of downtime a month costs about \$79,000 in lost sales (plus trust). Moving from 99.7% to 99.9% (43.8 minutes) would save about 87 minutes, or \$52,000 a month. If the engineering cost of the extra redundancy is less than that, it pays. The answer depends on the numbers, and the team should set the target by comparing cost and benefit.

An SLO and its budget. The team sets an objective: 99.9% of payment attempts succeed (excluding customer errors such as declined cards), measured monthly. The error budget is 43.8 minutes of equivalent failure. Mid-month, incidents have used 30 minutes, leaving 13.8. A policy agreed in advance: when less than 25% of the budget remains (below 10.96 minutes), pause risky launches and focus on reliability until the budget recovers. At 13.8 minutes the team may continue but with caution.

Idempotency and retries. Because a payment must not be charged twice when a request is retried, the payment call carries an idempotency key (C21.3). This is a reliability feature as well as a correctness one: it allows safe retries, which raises the effective success rate.

Reducing the chain. Ask what is on the critical path. Can the fraud check be asynchronous for low-risk transactions (C21.5)? Can the payment partner be backed up by a second provider? Each removal raises availability more than hardening a single component.

What this shows. Compounded availability explained why the feature was less reliable than its parts, the cost arithmetic set the target, and the error budget turned reliability into a planning tool.

6. Baseline, Strong, Standout

Prompt: "An analyst says weekly revenue by country was \$140,000, but finance says \$100,000. How would you work out the discrepancy?"

Baseline answer

"I'd ask the analyst and finance to get together and agree on one number, or check which one is right. Probably the analyst made a mistake, so I'd ask them to redo the query."

What is wrong: no method, assumes blame, and does not locate the cause. It would not find a systematic error.

Strong answer

"First I'd confirm they mean the same thing: same date range, same time zone, same definition of revenue (before or after refunds), same countries. If those match, the difference is likely in the query. I'd ask to see it and check what each table holds and how they are joined.

"A common cause of a number that is too high is a join that repeats rows: joining orders to order items counts each order once per item. I'd test it by checking whether the query's row count exceeds the number of orders, and by comparing a single order's total against its rows."

Standout answer

Everything in the Strong answer, plus:

"I'd use the 40% gap as a clue: \$140,000 against \$100,000 is 1.4 times, which suggests that some orders are being counted about twice on average, consistent with a join to a table with more than one row per order. I'd reproduce it on a tiny sample, say three orders, where I can compute the right answer by hand: if one order with two items gives its amount twice, the query has fanned out.

"I'd fix it by summing at the order grain before any join to items, and then check that the result matches the number finance trusts. I'd also check other usual suspects in order: time zone and date boundaries, orders with missing users dropped by the join, test orders, refunds and currency.

"The deeper fix is to prevent it happening again: agree one definition of revenue with finance and put it in a single shared, tested metric, with a reconciliation check against finance each week. I'd say this to the analyst as a shared process improvement, not a mistake.

"What I would not do: pick a number because it looks plausible, or assume which side is wrong before checking."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
Confirms definitions and looks at the query	Names the fan-out as a likely cause and tests it on a tiny sample
Checks the join and row counts	Reconciles to a trusted number and checks time zone, nulls and test data
Is neutral about who is wrong	Proposes a shared definition and an ongoing reconciliation

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge. Interview formats change [as of 2026-10].

Company lens: Apple [as of 2026-10]. Apple has described doing a large share of intelligence features on the device and has described a design for processing in the cloud with privacy protections for requests that need more capacity Public [V: Apple privacy page: on-device processing with Private Cloud Compute]. Third-party guides report that technical-fluency prompts at Apple concern on-device versus cloud processing, privacy and latency Inferred. Practical consequence: compare the two on latency, privacy, capability and cost, propose a hybrid with a clear disclosure, and state what stays on the device.

Company lens: Google [as of 2026-10]. Google has published its approach to site reliability, including service level objectives and error budgets, and its engineering is known for work on latency at very large scale Public [V: Google SRE book, 2016, and sre.google]. Candidates commonly report scale and performance prompts Inferred. Practical consequence: use percentiles, state an objective with a budget, and explain how a cache changes the tail.

Company lens: Meta [as of 2026-10]. Meta has published an account of its October 2021 outage, which began with a change to its backbone network Public [V: Meta engineering post of 5 October 2021] (section 10). Candidates commonly report prompts on reliability, rollout and what happens when a safety mechanism misfires Inferred . Practical consequence: discuss blast radius, independence of recovery tools, and drills.

Company lens: Microsoft [as of 2026-10]. Microsoft operates cloud services and enterprise products where reliability, compliance and administrator control matter, and publishes incident reviews Public [V: Azure status history publishes post incident reviews] . Candidates commonly report prompts on security, compliance and service reliability in enterprise settings Inferred . Practical consequence: cover authentication and authorization, audit logs, data residency and availability commitments.

Company lens: Amazon [as of 2026-10]. Amazon runs a cloud platform and an online store where small increases in latency have been described as costly, and it emphasizes frugality Public [V: AWS launch account; Frugality is a Leadership Principle; the latency claim was not rechecked] . Candidates commonly report prompts that weigh speed, cost per request and reliability Inferred . Practical consequence: state cost per request at expected volume, and use measured effects of latency, not a remembered statistic.

Company lens: Netflix [as of 2026-10]. Netflix has published on running a large service on a cloud provider and on testing resilience by injecting failures Public [V: Netflix Tech Blog posts on experimentation and resilience testing] . Candidates commonly report prompts on reliability and graceful degradation Inferred . Practical consequence: describe what the user sees when a dependency fails (a fallback, not a blank screen), and how you would test it.

Company lens: Stripe [as of 2026-10]. Stripe's API documentation describes idempotency keys so that retried requests do not repeat an action, and the company is widely described as emphasizing API quality Public [V: Stripe API documentation on idempotency, webhooks and versioning] . Candidates report prompts on API design and reliability Inferred . Practical consequence: raise idempotency, versioning and clear errors whenever money or irreversible actions are involved.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	Jargon used incorrectly	A term you cannot explain simply	Use plain words; say when you are unsure
2	Averages instead of percentiles	"The average is fine"	Ask for the 95th and 99th percentile
3	"Just cache it"	No thought of staleness	Ask how fresh the data must be
4	No idempotency on actions	Retries could repeat a charge or message	Ask what happens if it is sent twice
5	Trusting a joined total	A number that exceeds a known figure	Check the grain and the join; match a known total
6	Ignoring dependency compounding	Each service "is 99.9%"	Multiply the chain; shorten it
7	One-factor on-device or cloud choice	"Cloud is faster"	Compare latency, privacy, cost, capability, offline
8	Ignoring the poor-network and old-device case	Tested only on the team's phones	Test on representative devices and networks
9	Telling engineers how to build	"Use a queue"	State what must be true; ask for options
10	Ignoring cost per request	Cost to build only	Ask for cost at expected volume

9. Mental models applied

Four models, each tied to a decision in this chapter. `[[second-order-effect]]` is from C02.

Amdahl's law (`MM-amdahl`). The overall speedup from improving one part of a process is limited by the share of the whole that part represents `[V: Amdahl's law, a mathematical identity]`. If a hop is 5% of the time, making it free gains at most 5%. Use it to choose what to fix: find the largest contributor at the percentile that matters (Worked example 1).

Single point of failure (`MM-spoof`). A component whose failure stops the whole system. Look for one in every request path and every recovery path. CS-C21-02 shows a single point that was hidden: the system's own recovery tools depended on the network that had failed.

Leaky abstractions (`MM-leaky-abstractions`). Simplifying layers hide detail until the detail matters; a simple API hides a network that can fail, a database that can be slow, a cache that can be stale `[V: Spolsky, The Law of Leaky Abstractions, 2002]`. A PM who knows the layer beneath the one they use is less surprised.

The fallacies of distributed computing (`MM-fallacies`). A list of false assumptions engineers make about networked systems: that the network is reliable, that latency is zero, that bandwidth is infinite, that the network is secure `[V: Fallacies of distributed computing (eight assumptions), as listed in software-engineering law references]`. Each is a prompt for a product question: what does the user see when the network is slow, or down?

Engineering, design and data-science perspectives

- **Engineering.** Engineers want product requirements that state what must be true (latency target, freshness, availability) and leave the how to them. Ask the ten questions in FW-C21-02; thank them for risks surfaced early; avoid specifying a solution in the spec.
- **Design and research.** Designers care about what the user sees while the system is working or failing: loading, error and offline states (C10.5). Share the latency budget with them so that perceived speed (progress, skeleton screens) is designed in.
- **Data science.** Data scientists depend on pipelines and instrumentation. Agree the event definitions, the freshness needed and who owns each transformation, and ask for reconciliation checks against trusted totals.
- **Security, privacy and legal.** They need to know what data is collected, where it is processed and who can see it. Bring them into the on-device versus cloud decision and the retention design before the build.

Checklist: before you commit to a technical approach

1. Can I draw the request path and say where time and failure concentrate?
2. Have I asked for percentiles, not averages?
3. Do I know the contracts (API, schema) that others depend on, and what happens if a request is sent twice?
4. Have I asked how fresh the data must be before accepting a cache or a copy?
5. Do I know where each number comes from, and have I checked a joined total against a known figure?
6. Have I compounded the availability of the dependency chain and set a target with a budget?
7. For any feature that processes personal data, have I compared on-device and cloud and said what leaves the device?
8. Have I asked engineering for the riskiest part, the cost at expected volume, and the rollback plan?

10. Case studies

CS-C21-01 · The search page that scanned two million rows · Illustrative · Beginner

Fictional composite. Every number is invented (invented).

Situation. A marketplace's search page takes 3.1 seconds to load on average. Support hears that it is slow, and the team has proposed a full redesign of the front end. The PM asks engineering where the time goes before approving the project.

Constraints. A small team, a quarter of capacity, and a belief that "the page is heavy."

Data. Engineers instrument the request path. Of the 3.1 seconds: name lookup and connection 0.3, transfer and rendering 0.5, application logic 0.2, and the database query that finds matching listings 2.1 seconds. The query filters a table of 2,000,000 listings by category and city, and the table has no index on those columns, so the database examines every row. A check of the query plan confirms a full table scan.

Problem definition. As an observable problem: "Most of the page's load time is one database query that reads every row because the lookup column has no index; the front end is a small part of the delay."

Analysis. A redesign of the front end could at best save part of the 0.5 seconds of transfer and rendering (about 16% of the page), while the query accounts for 68% (Amdahl's law). Adding an index on category and city lets the database find matching listings directly. Engineers estimate two days of work, plus a few hours to build the index on a large table without blocking writes, and a small cost in write time and storage. With the index, the query takes 12 milliseconds instead of 2,100 (a 175-fold improvement), and the page load falls to $0.3 + 0.5 + 0.2 + 0.012 =$ about 1.0 second, which engineers measure at 0.8 to 1.0 seconds.

Options.

Option	Optimizes	Costs
A. Redesign the front end	Look and feel; maybe 0.2 s	A quarter of capacity; the main delay remains
B. Add the index and fix other slow queries	The largest contributor	Two days; slight write cost
C. Add a cache in front of the query	Repeated searches	Staleness; complexity; the first search stays slow

Tradeoffs. B is the cheapest and fixes the biggest cause. C helps repeated searches and leaves the first one slow. A spends the most and changes the least.

Decision. Choose B, and measure percentiles before and after (the 95th percentile, not only the average). Primary measure: 95th-percentile page load time; guardrails: write latency on listing creation, and search result correctness. Because speed changes may affect orders for other reasons, run the change behind a flag with a holdback group to measure the effect on conversion (C14).

Outcome. In this fictional scenario the 95th-percentile load time fell from 5.4 to 1.7 seconds, and the conversion rate on search sessions rose from 2.1% to 2.5% in the exposed group against a flat holdback (invented). On 1,000,000 search sessions a month, 0.4 points is 4,000 more orders, at an average of \$30, about \$120,000 a month. The redesign was deferred, and its scope was reduced to what testing showed to matter.

Alternative outcomes. If the indexed query had still been slow at the tail, the team would have looked at the second-largest contributor (transfer on slow networks) next. If conversion had not moved, the faster page would still be worth keeping, but the case for further speed work would weaken. The lesson: find which hop dominates before choosing the project.

CS-C21-02 · Facebook, October 2021: when the front door disappeared · Verified · Advanced

Sources (types): Meta's own engineering account of the outage, "More details about the October 4 outage" by Santosh Janardhan (5 October 2021), read in full (primary, for the cause and the design of the systems involved); Cloudflare's analysis of the same day, "Understanding how Facebook disappeared from the Internet" (4 October 2021), and ThousandEyes' outage analysis, read through excerpts, for the external timing; and trade-press reporting (The Register, ComputerWeekly) of Meta's post. The analysis is the author's. The sources do not reveal Meta's internal records. Re-read in October 2026.

Situation. On 4 October 2021, from about 15:40 UTC, Facebook, Instagram, WhatsApp and related services became unreachable worldwide. Cloudflare saw Facebook's routes return from about 21:20 UTC, ThousandEyes measured the disruption until about 22:45 UTC, and press reports describe an outage of about six hours, so the length depends on what is measured [V: Cloudflare; ThousandEyes; press]. Meta's account says the outage was triggered by the system that manages its global backbone network capacity. During routine maintenance, a command intended to assess the availability of global backbone capacity unintentionally took down all the connections in the backbone, disconnecting Meta's data centers from one another and from the internet. Meta says its systems are designed to audit such commands, but a bug in the audit tool prevented it from properly stopping this one [V: Meta's post]. Meta's account adds that its smaller edge locations host DNS servers, which disable their routing advertisements if they cannot reach the data centers (a sign of an unhealthy network connection). With the whole backbone out, all of them withdrew those advertisements. The DNS servers remained operational but became unreachable, so the rest of the internet could not find Meta's servers.

Constraints. A very large, tightly integrated network in which the systems used to diagnose and repair problems may depend on the same network that has failed. Meta says its data centers are designed with high levels of physical and system security, which slowed engineers who had to be sent on site, and that recovery had to bring load back gradually because dips in power use of tens of megawatts at individual data centers made a sudden surge risky [V: Meta's post]. Meta also says it had long run "storm" drills that take a service, data center or region offline, but had never run one that simulated its global backbone going offline [V: Meta's post].

Data. As reported: the date and approximate start; the trigger (a command during maintenance); the effect (all backbone connections down); the design of the DNS servers' health check (withdraw routes when unhealthy); the result (DNS unreachable though operational); a duration of about six hours; and a restoration that required engineers to go on site, because Meta says its primary and out-of-band network access was down and the loss of DNS broke many internal tools [V: Meta's post].

Problem definition. *The author's reading.* A change made to one system (backbone capacity management) had a global effect, and a safety mechanism in another system (DNS route withdrawal) turned a network outage into a total disappearance, while the tools to recover were affected by the same failure. As an observable problem: "A command meant to be read-only for capacity took down the backbone, and everything that depended on it, including the way back, went with it."

Analysis. *The author's analysis.* The request path (FW-C21-01): a user's device asks DNS for the address of the service; DNS answers; the device connects across the internet to a data center. Meta's failure hit the first hop in an unusual way: not by crashing the DNS servers but by making them unreachable, because the routes to them were withdrawn. The user's experience was that the name did not resolve and nothing loaded.

A safety mechanism as a single point of failure. The health check made sense locally: a DNS server that cannot reach the data centers should not answer, because it might give outdated answers. At global scale the same rule meant that when every location lost its backend at once, every location removed itself. A rule that protects against a local failure can amplify a global one. The PM-level lesson is to ask, for every protective mechanism, what happens if it triggers everywhere at once.

Blast radius and independence (C17.2). The change was global in effect. The recovery path shared fate with the failure: Meta says the loss of DNS broke many internal tools and that both its primary and its out-of-band network access were down (some press reports add that physical access systems were affected, which Meta's post does not say). A principle: recovery tools should not depend on the thing they recover, and even an out-of-band path needs its own independence.

Rollback and drills (C17.5). Meta says it had never run a storm drill that simulated losing its global backbone, although its other storm drills prepared it to bring load back carefully, and recovery could not be undone with a remote command. Staged rollout of changes to critical systems, an audit step that can prevent a command from taking down everything, and regular drills for the worst case are the practices that this case argues for. The postmortem culture (C17.5) is visible in Meta's published account, which explained the cause in plain terms.

What the evidence does not show. The internal reasoning behind the design, the exact tool that failed to prevent the command, and details of the physical recovery. The case supports a structural analysis only, and no judgment of any individual.

Options. *The author's reconstruction of choices a team could have faced; not a record of Meta's deliberations.*

Option	Optimizes	Costs
A. Keep the DNS withdrawal rule as designed	Prevents stale or wrong answers in a local failure	Amplifies a global failure
B. Keep the rule but add a global guard: if all locations would withdraw at once, keep some answering	Protects against both	Complexity; a harder rule to reason about

Option	Optimizes	Costs
C. Separate recovery tools and access onto an independent network	A way back when the main network fails	Cost of a second network and its upkeep
D. Gate backbone commands with a simulation or a staged apply, and drill backbone loss	Prevents the trigger	Slower maintenance; engineering effort

Tradeoffs. Each option adds cost or complexity to guard against a rare event. The cost of the event (billions of users, six hours) is the comparison.

Decision. As documented, Meta restored the backbone and brought services back gradually, and its account says it will review the incident and strengthen testing, drills and resilience, and look for ways to simulate events like this [V: Meta's post] .

Outcome. Reported: services restored on the evening of 4 October (Cloudflare saw routes return from about 21:20 UTC; press reports say about six hours), with Meta's detailed explanation published on 5 October [V: Meta's post; Cloudflare] .

Alternative outcomes. *Inference.* With an independent recovery path the outage might have been shorter; with a global guard on the DNS rule the services might have stayed partly reachable. A reader should avoid hindsight: the individual pieces were reasonable, and the failure came from how they interacted. The method to carry away: draw the request path and the recovery path, ask what each protective rule does if it triggers everywhere, keep the way back independent of the thing that can fail, and rehearse the worst case.

11. Interview questions and model answers

Seven questions: two Beginner, two Intermediate, three Hard. Model answers are **Illustrative** (composite candidates), every figure is an assumption, and the arithmetic was computed. Each is written against the six behaviors in section 1. Where a real company's product is the setting, the answer reasons from public information and states no internal fact. The answers are fuller than a first pass in a real round.

IQ-C21-01 · Beginner · technical · TE

Prompt. What happens when you open an app and your feed loads?

Rubric.

- **Baseline:** "The app asks the server for the data and shows it."
- **Strong:** Walks the path: device, name lookup, connection, server, database, response, rendering.
- **Standout:** Says where time and failure concentrate, mentions caching and percentiles, and ties the path to what the user sees when a hop fails.

Model answer. I'll walk the path in the order it happens, and say where it can go wrong.

First, the app on my phone decides what to ask for: the latest items for my account. It needs the address of the company's server, so it asks the Domain Name System, which turns a name into a numeric address. A copy of recent answers is kept on the phone or the network, so this is usually fast. Then the phone opens a secure connection to the server's front door, which is usually a load balancer that spreads requests among many servers, often behind a content delivery network that already holds common files near me.

The request reaches an application server, which checks who I am, then asks other services and a database for what goes into my feed: my follows, recent posts, ranking, maybe ads. It may use a cache for popular items. It assembles a response and sends it back, and my phone draws it, loading images as I scroll.

Where time goes. On a good network, the server and database work dominates; on a poor network, connection setup and transfer dominate; on an old phone, drawing can. So when someone says it is slow I ask which hop and at which percentile, because the median can look fine while one user in twenty waits several seconds.

Where it fails, and what the user sees. If the name lookup fails, nothing loads. If the database is slow, the feed is slow. If one service (say ads) is down, a well-designed app shows the feed without ads and not a blank screen: graceful degradation. The thing I would check is what each hop shows the user when it fails.

Follow-ups.

- **Q.** Where would you look first if the feed were slow, and why not the average? **A.** "I would look at the 95th-percentile load time, split by hop: name lookup and connection, server compute, database, transfer and rendering. The average hides the slow minority, who are often the ones complaining, on weak networks and older phones. If the median is 1.2 seconds and the 95th percentile is 6 seconds, the typical user is fine and one in twenty waits five times as long. I would find which hop contributes most to the slow group and ask engineering what is feasible there before proposing any change."
- **Q.** What does the user see when a hop fails, and what should they see? **A.** "Often a blank screen or a spinner that never ends, which feels like the app is broken. A better design shows the last cached feed with a small notice that it could not refresh, keeps a retry in one tap and loads parts independently so that a failed image does not block text. I would measure the error rate per hop and the share of sessions that show a degraded state, and I would set a rule that a failure in a non-essential service must not block the core feed."

IQ-C21-02 · Beginner · technical · TE

Prompt. Explain to a non-technical stakeholder what an API is.

Rubric.

- **Baseline:** "It's how programs talk to each other."
- **Strong:** Gives a plain definition, an analogy and a product example.
- **Standout:** Adds the contract idea, versioning, and why it matters to the business (partners, speed, risk), and says one tradeoff.

Model answer. An API is a menu of requests that one piece of software offers to another. Like a restaurant menu, it says what you can ask for, in what form, and what you will get back, without you needing to know how the kitchen works.

An example from a product: when a travel app shows the weather, it does not measure the weather. It sends a request to a weather company's API, "forecast for this city tomorrow," and gets back a small, structured answer that the app displays. When you pay in an online shop, the shop does not handle your card; it calls a payment company's API, "charge this amount," and gets back "approved" or "declined."

Why this matters for the business. First, it lets us build on others' work and lets others build on ours, which is how platforms grow. Second, an API is a *contract*: once partners build on it, we cannot change it casually, so changing it is a planned, announced process with versions. Third, it is a place where reliability and security are decided: every request is a door, and each needs a lock, a limit on how often it can be used, and monitoring.

One tradeoff to know: making an API simple for partners means committing to keep it working, which limits how freely our own engineers can reshape the system behind it. So I ask what the contract is, who depends on it, and what it costs to change.

Follow-ups.

- **Q.** What is versioning, and why does it matter to the business? **A.** "A version is a labeled form of the contract, so that changes can be introduced without breaking people who depend on the old one. Adding a field is usually safe; removing or renaming one breaks callers. Partners build products on an API, so a breaking change costs them engineering time and may make them leave. The business reason is trust and speed: a stable, versioned API lets partners integrate quickly and invest, and a careless change can cost more than the feature was worth. I would ask how long old versions are supported and how partners are told."
- **Q.** What is one tradeoff of offering an API? **A.** "It speeds growth through partners and creates obligations. Once others rely on the API, the company must keep it stable, secure and documented, which limits how freely it can change the internals, and it exposes a surface that can be abused, so limits on request rates and authentication are needed. A private interface can change tomorrow; a public one must be supported for years. The decision to open an API should weigh partner value against the maintenance and security cost."

IQ-C21-03 · Intermediate · technical · TE · Apple

Prompt. Explain on-device versus cloud processing for a voice assistant, including privacy and latency.

Rubric.

- **Baseline:** "On-device is more private and cloud is more powerful."
- **Strong:** Compares latency, privacy, capability, cost and offline use.
- **Standout:** Quantifies latency on good and poor networks, proposes a hybrid with routing and disclosure, and treats sensitive requests specially.

Model answer. I know the product only from public information; nothing here describes any real system. *On-device processing* means the phone does the work itself; *cloud processing* means it sends the audio or its transcript to a server and gets the answer back.

Privacy. On the device, the data never leaves, which is the strongest guarantee and a trust signal. In the cloud, the data is sent and handled by a server, so the company must justify what it collects, how long it keeps it, and who can see it. Users should be told which happens and given control.

Latency. The device has no network wait, but its processor is slower than a server's. Suppose (invented) on-device compute takes 150 ms; the cloud computes in 40 ms but needs a round trip of 60 ms and 20 ms to upload on a good connection, 120 ms in all; on a poor connection with a 400 ms round trip it is 460 ms, and offline it fails. So the cloud is slightly faster when the network is good and much slower when it is not; on-device is predictable.

Capability. The cloud can run larger models and use more data, so it handles hard requests better; the device is limited by memory and battery. Suppose easy requests (80% of traffic) are handled at about 95% accuracy on the device and hard ones at 70%, against 90% for hard requests in the cloud.

Cost. The cloud costs the company per request (say \$0.002), which at a billion requests a day is \$2 million a day; the device shifts cost to the user's battery and the model's size on the phone.

My recommendation. A hybrid: do the easy 80% on the device and send only the hard 20% to the cloud, with a short routing step. In my numbers that gives an average latency near 146 ms, accuracy near 94% (against 90% for device only and about 95% for cloud only), and a fifth of the cloud cost. I would keep sensitive categories on the device whatever their difficulty, tell users plainly what is sent, and measure the share routed to the cloud. What would change my view: on-device quality closing the gap, which would move more work to the device.

Company note. Apple is the setting only: Inferred from third-party reports that technical-fluency prompts at Apple concern on-device processing and privacy [unverified: candidate-reported]. No claim is made about any real product's design, numbers or data handling.

Follow-ups.

- **Q.** How would you quantify the latency difference on good and poor networks? **A.** "With round, invented figures. On-device recognition might respond in 150 to 300 milliseconds regardless of the network. A cloud round trip might take 400 to 800 milliseconds on a good connection and two to four seconds on a poor one, or fail. So for short commands such as set a timer, on-device is both faster and more reliable, and for complex requests that need a large model or fresh data, the cloud may be necessary. A hybrid routes simple intents on-device and complex ones to the cloud, and tells the user when something is sent."
- **Q.** How would you treat sensitive requests? **A.** "With the strongest protection by default. Requests about health, finance, messages or location would be handled on the device where possible; where the cloud is needed, the audio or text would be encrypted in

transit, kept only as long as needed to answer, not used for training without consent, and clearly indicated to the user. I would give a setting to keep everything on-device with a clear statement of what stops working, and have privacy review the routing rules. The measure is the share of sensitive requests handled locally and complaints about privacy."

IQ-C21-04 · Intermediate · technical · TE

Prompt. Customers say a page is slow. How do you work with engineers to find out why and what to do?

Rubric.

- **Baseline:** "Ask engineering to make it faster."
- **Strong:** Asks for measurements by hop and percentile, finds the biggest contributor, and decides on a fix.
- **Standout:** Uses a latency budget, checks the slow devices and networks, considers side effects and measures the effect on users.

Model answer. I'd turn "slow" into numbers before asking for any fix. I'd ask engineering for the load time at the median and the 95th percentile, split by hop: name lookup and connection, server compute, database, transfer, and rendering. I'd also ask who is slow: which devices, networks and countries, because averages hide the slow minority, and the customers who complain are often in it.

Then I'd find the largest contributor at the percentile that matters. Suppose the tail is 5.2 seconds and 2.0 of it is one database query. By Amdahl's law, speeding up a hop that is 5% of the time can save at most 5%, so I would not start with the front-end polish. I'd ask the engineer what is slow about that query; a common answer is a missing index, a cheap fix. I'd also ask about image sizes (transfer) and whether any result could be cached and how stale it may be.

I'd put a target in writing: a 95th-percentile load time under 3 seconds on a mid-range phone on a typical mobile connection, not "faster." I'd ask what the fixes cost, what they risk (a cache can show stale data, an index slows writes slightly), and how we will roll them out with a flag and a holdback so that we can measure the effect on bounce and conversion, not just on load time.

Finally I'd set a guardrail so that speed does not come at the cost of correctness, and I'd re-measure after each change. What I would not do: assign a rewrite without knowing where the time goes.

Follow-ups.

- **Q.** What is a latency budget and how would you use it? **A.** "A target for the total time split across the steps that make it up. For a page with a 2.5-second target at the 95th percentile, I might budget 400 milliseconds for network and connection, 300 for the server, 200 for the database, 800 for rendering and 800 as margin. Each team then owns a slice, and when the page exceeds the target, the budget shows which slice overran. It also gives the PM a way to talk about tradeoffs: adding a feature that costs 150 milliseconds of rendering uses a fifth of that slice and must be weighed against its value."

- **Q.** How would you measure the effect of the fix and its side effects? **A.** "Release the fix behind a flag to a random group and compare the 95th-percentile load time by device and network, plus conversion and bounce rate. Check side effects: a cache that speeds the page may show stale content, a lighter page may drop features, and a compressed image may look worse. The guardrails are error rates and cost per request. If the median improves and the 95th percentile does not, the slowest users have not been helped, so I would find what is different about them."

IQ-C21-05 · Hard · technical · TE

Prompt. Users say that after saving a change to their profile, they sometimes see the old version. What could cause this, and what would you decide?

Rubric.

- **Baseline:** "It's a bug; ask engineering to fix it."
- **Strong:** Names caching and copies of data as causes, and explains staleness.
- **Standout:** Distinguishes causes (browser or app cache, server cache, read replica lag), sizes the problem, proposes read-your-own-writes and design options with tradeoffs, and sets a measure.

Model answer. This is a classic symptom of data that exists in more than one place, so one copy is updated before the others. I'd list where copies live, because the cause depends on which one is stale.

Possible causes. (1) *The client's cache:* the app or browser keeps the old profile and shows it until it refreshes. (2) *A server-side cache:* the application stores a copy of the profile for speed, and the write did not clear it, or the cache's lifetime has not expired. (3) *Read replicas:* to handle load, the database keeps copies that serve reads, and a copy can lag behind the main one by a short time (eventual consistency); if the page after saving reads from a lagging copy, the user sees the old data. (4) *A queue:* if the write is processed asynchronously, the change may not have been applied yet.

Size it. Ask for the rate and timing. Suppose (invented) 200,000 profile saves a day, and 5% of users view their profile within two seconds of saving, which is about 10,000 users a day who could see stale data if replica lag is about two seconds. That is a visible, trust-damaging number: people think their change was lost, and some will save again or contact support.

Options. (A) *Read-your-own-writes:* for a few seconds after a user writes, route their reads to the main database or the fresh cache. Cost: a small extra load on the main database (here 10,000 reads a day, negligible) and a little logic. (B) *Show the new data immediately on the client,* then reconcile (optimistic update). Cost: a brief inconsistency if the save fails, which needs a clear error. (C) *Shorten cache lifetimes or clear the cache on write.* Cost: less speed benefit. (D) *Accept it:* acceptable for data like a follower count, not for something the user just changed.

Decision. A and B together for user-initiated changes (the user must see their own change at once), and keep the long cache lifetimes for other users' views of the profile, where a delay of seconds is harmless. Measure: the rate of "I saved but see the old value" support contacts

and a client check of whether the displayed value matches the saved one. The lesson to say aloud: speed comes from copies, and copies can disagree, so the product decides where staleness is acceptable.

Follow-ups.

- **Q.** What design options deliver read-your-own-writes, and what do they cost? **A.** "Three common ones. After a write, read that user's data from the primary copy for a few seconds, which costs extra load on the primary. Return the updated record in the response to the write and update the client's cache directly, which needs client changes. Attach a version number to the write and refuse to show data older than that version, which adds complexity. Each trades consistency for load or engineering effort. For a profile that people check right after editing, the second is often the cheapest and most visible fix."
- **Q.** How would you size the problem and decide how much to invest? **A.** "By measuring it. Instrument the share of reads within five seconds of a write that return the old version, and the number of support contacts about changes not saving. If 2% of edits show a stale profile at first view, that is thousands of confusing moments and worth fixing; if it is 0.01%, a copy change such as Your changes are saved and may take a moment to appear may be enough. I would set a target, for example under 0.1% of post-write reads stale, and review it after the fix."

IQ-C21-06 · Hard · technical · TE

Prompt. A messaging app wants to set reliability targets. How would you approach it, and what would you commit to for sending messages versus loading media?

Rubric.

- **Baseline:** "As reliable as possible, five nines."
- **Strong:** Sets different targets for different functions, using the cost of failure.
- **Standout:** Computes downtime for each target, compounds dependencies, defines an SLO and error budget with a policy, and designs graceful degradation and retries with idempotency.

Model answer. I'd begin with what a failure costs the user, because a single target for everything wastes effort. Sending a message is the core promise; if it fails, people lose trust fast. Loading a photo that arrives a moment late is an annoyance.

Targets and what they mean. A month has about 43,834 minutes. A target of 99.95% for sending allows $0.0005 \times 43,834 =$ about 22 minutes of failure a month. A target of 99.5% for loading media allows 219 minutes, about 3.6 hours. I would not say "five nines" without a price: each extra nine costs more redundancy and complexity, so I'd ask engineering what 99.95% costs against 99.9% (44 minutes) and compare it with what an extra 22 minutes of failures cost in trust.

Dependencies. Sending depends on several services (authentication, the message store, the delivery service); in a chain, availability multiplies. Three services at 99.95%, 99.95% and 99.9% give about 99.8%, which is 88 minutes of failure, worse than the target. So either the chain must be shortened (can delivery be asynchronous?) or each part must be better.

Design for the failure. Make sending resilient: save the message on the device, show it as "sending," and retry in the background until the server confirms, with an idempotency key so that a retry never produces a duplicate message. For media, load a low-resolution preview first and fetch the full file when possible, so a slow media service degrades the experience without breaking the conversation.

SLO and error budget. Define the objective in terms users feel: 99.95% of send attempts are accepted within two seconds, measured monthly. The budget is about 22 minutes. A policy decided in advance: if less than a quarter of the budget remains, pause risky launches and focus on reliability; if the budget is spent, only fixes ship until it recovers.

Measures. Successful sends per attempt, the 95th and 99th percentile send time, duplicate message rate (should be zero), and how long messages sit in "sending." What would change my approach: evidence that outages in one region hurt a particular market much more, which would justify regional redundancy there.

Follow-ups.

- **Q.** What do 99.95% and 99.5% mean, and why would the targets differ? **A.** "In a month of about 43,834 minutes, 99.95% allows about 22 minutes of failure and 99.5% allows about 219 minutes. Sending a message is the core promise, and a failure means a lost conversation, so it justifies the tighter target and the engineering cost of reaching it. Loading media that arrives a moment late is a smaller harm, so a looser target saves effort for work that matters more. Each target also has a price: going from 99.5% to 99.95% typically needs redundancy and more careful releases, and I would ask engineering what it costs."
- **Q.** What error-budget policy and retry design would you propose? **A.** "The budget is the allowed failure, here 22 minutes a month for sending. A policy: if half the budget is used in the first week, freeze risky releases and focus on reliability until it recovers. For retries, every send carries an idempotency key, so that a retry after a timeout does not create a duplicate message. If sending fails, the app queues the message on the device, shows a clear sending state and retries with increasing delays. This is graceful degradation: the user sees that the message is pending, not a silent loss."

IQ-C21-07 · Hard · technical · TE

Prompt. Engineering says a change requires a database migration with two hours of downtime. How do you evaluate it?

Rubric.

- **Baseline:** "Do it at night and tell customers," or "downtime is unacceptable."
- **Strong:** Weighs the cost of downtime against the cost of avoiding it.

- **Standout:** Asks the ten questions, computes cost and the effect on the reliability budget and contracts, knows the expand-and-contract alternative, and plans rollback and communication.

Model answer. I'd treat it as a cost and risk decision and start with questions to engineering (FW-C21-02): why is downtime needed, what changes in the data, what is the riskiest step, and what is the rollback if it fails halfway? Is there a way to migrate without downtime?

Cost of the downtime. Suppose (invented) the product earns \$20,000 an hour at peak. A two-hour window at 03:00, when traffic is about 15% of peak, costs about $2 \times \$20,000 \times 0.15 = \$6,000$ in lost sales, plus support contacts and some trust. That is small.

Cost of avoiding it. The alternative is a staged "expand and contract" migration: add the new structure beside the old, write to both, copy the old data in the background, switch reads to the new, then remove the old. It avoids downtime and adds a few weeks; suppose three person-weeks at \$15,000 each, \$45,000, plus more moving parts and more risk of a subtle bug during the dual-write period.

Reliability budget and contracts. At a 99.9% monthly objective the budget is 43.8 minutes; two hours is 120 minutes, about 2.7 times the budget. If customers have service agreements, check whether planned maintenance is excluded and how much notice is required; some enterprise contracts forbid planned downtime in business hours or require notice. So the cash comparison (\$6,000 against \$45,000) is not the whole story: the budget and contracts may force the staged approach.

Plan either way. Rehearse the migration on a copy of production data and time it (two hours is an estimate, not a measurement). Set a go or no-go point and a rollback that is tested. Take a verified backup. Choose the quietest window and tell customers in advance with a status page. Have support ready. Define a trigger for abort (for example, if the copy is not 60% complete by the midpoint).

Decision. If the contracts allow notice and the window is quiet, accept the planned two hours, with the rehearsal and rollback plan. If contracts or the budget forbid it, or if the data is critical and rollback is hard, fund the staged migration. What would change my mind: a rehearsal showing the migration takes six hours, which would make the staged route clearly better.

Follow-ups.

- **Q.** What questions would you ask engineering, and what is the expand-and-contract alternative? **A.** "Why downtime is needed, what changes in the data, how much data and how long it takes (a range), the riskiest step, how it was tested on production-sized data, the rollback and the point of no return, dependencies on other systems, and whether there is a way to avoid downtime. The alternative, expand and contract, adds the new structure alongside the old one, writes to both, backfills existing data, switches reads to the new structure, and only then removes the old one. It takes more steps and time and often avoids downtime entirely, so I would ask whether the extra weeks cost less than the two hours."

- **Q.** How do you plan the rollback and the communication? **A.** "With checkpoints and a verified backup. Before starting, confirm that a restore from backup works and set go or no-go checks at each stage, with the point of no return named. Announce a week before through the status page and direct messages, schedule it in the lowest-traffic window, and prepare a message for each possible outcome. Cost: at \$20,000 an hour at peak and 15% of peak at 3 a.m., two hours costs about \$6,000 in lost revenue, but if customers have a 99.9% availability commitment, which allows about 44 minutes a month, two hours breaches it, and service credits or contract terms may cost more. I would check contracts before choosing the window."

12. Exercises

Ten exercises. Each has a model solution and an alternative. Figures are assumptions unless marked, and the arithmetic was computed.

EX-C21-01 · Draw the request path

Problem. A user taps "Send \$20" in a payments app. Draw the request path in hops, say what can fail at each, what the user should see, and add up a latency budget given: lookup 50 ms, connection 150 ms, server and fraud check 250 ms, database write 120 ms, response and render 180 ms.

Context. A payments feature review.

Data. As above.

Constraints. Seven minutes.

Expected approach. FW-C21-01.

Solution. Hops: device, name lookup, secure connection, load balancer, application server, fraud-check service, database, payment partner, response. Failures: lookup or connection (nothing happens; show "check your connection" and keep the amount); server or fraud check slow or down (show "processing," retry with the same idempotency key); database write fails (do not show success; the payment must not be marked sent); partner declines (show the reason); response lost after success (the user may tap again, so an idempotency key must make the repeat safe and show the original result). Budget: $50 + 150 + 250 + 120 + 180 = 750$ ms, under a one-second target; the server and fraud step is the largest at 250 ms.

Alternative solution. Run the fraud check asynchronously for low-risk amounts to bring the total under 500 ms.

Tradeoffs. Asynchronous checks lower latency and raise the risk of completing a payment that is later reversed.

EX-C21-02 · Explain it in 90 seconds

Problem. Write a 90-second plain-language explanation (definition, analogy, product example, one tradeoff) for each of: (a) a cache; (b) a database index; (c) an API.

Context. A stakeholder meeting.

Data. None.

Constraints. Eight minutes.

Expected approach. Section 4, explain-a-concept type.

Solution. (a) A cache keeps a copy of data close to where it is needed, like keeping popular books on a front shelf; a shopping page loads faster because product details are served from the copy; the tradeoff is that the copy can be out of date. (b) An index lets the database find rows without reading every one, like the index of a book; a search on category and city goes from reading two million rows to a direct lookup; the tradeoff is extra storage and slightly slower writes. (c) An API is a menu of requests one program offers another; a weather app asks a weather service for tomorrow's forecast; the tradeoff is that once others build on it, it cannot be changed casually.

Alternative solution. Use an analogy from your own product's domain.

Tradeoffs. An analogy simplifies and can mislead; say where it stops fitting.

EX-C21-03 · Percentiles

Problem. Twenty response times in milliseconds: 120, 130, 140, 150, 150, 160, 170, 180, 190, 200, 210, 220, 240, 260, 300, 350, 420, 800, 1,500, 3,200. Compute the mean, the median, the 95th percentile (nearest-rank method) and the maximum, and say which you would report.

Context. A latency review.

Data. As above.

Constraints. Five minutes.

Expected approach. C21.2 and C12.

Solution. The mean is $9,090 \div 20 = 454.5$ ms. The median is the average of the 10th and 11th values: $(200 + 210) \div 2 = 205$ ms. The 95th percentile by nearest rank is the 19th value: 1,500 ms. The maximum is 3,200 ms. The mean (454.5) is more than double the median (205) because a few slow responses pull it up; it describes nobody's experience. Report the median and the 95th percentile, and set the target on the 95th (and the 99th for critical paths).

Alternative solution. Report the share of responses over a threshold (for example, 3 of 20 take over 400 ms, or 15%).

Tradeoffs. Small samples make percentiles noisy; use many requests in practice.

EX-C21-04 · Cache arithmetic

Problem. A cache answers in 5 ms and the backend in 150 ms. Compute the average latency at hit rates of 80% and 95%. At a 96% hit rate, what are the 95th and 99th percentile latencies?

Context. Sizing a cache.

Data. As above.

Constraints. Five minutes.

Expected approach. Worked example 3.

Solution. At 80%: $0.8 \times 5 + 0.2 \times 150 = 34$ ms. At 95%: $0.95 \times 5 + 0.05 \times 150 = 12.25$ ms. At 96%, 4% of requests miss: the 95th percentile falls within the hits (96% are fast), so it is about 5 ms; the 99th percentile falls within the misses, so it is 150 ms. The average improves steadily with hit rate, but the 99th percentile stays at the backend's speed until the hit rate exceeds 99%. If the target is on the tail, the cache must be very effective or the backend must be faster.

Alternative solution. Improve the backend's own latency for misses (an index, a faster query) in addition to raising the hit rate.

Tradeoffs. A larger cache costs memory; compare with the value of the faster tail.

EX-C21-05 · A LEFT JOIN that behaves like an inner join

Problem. Tables: users u1, u2, u3. Orders: o1 (user u1, paid), o2 (u1, refunded), o3 (u3, paid). Query: `SELECT u.id, COUNT(o.id) FROM users u LEFT JOIN orders o ON o.user_id = u.id WHERE o.status = 'paid' GROUP BY u.id`. What does it return? What did the analyst intend (paid orders per user, including zero)? How do you fix it?

Context. Reading a query.

Data. As above.

Constraints. Six minutes.

Expected approach. FW-C21-04.

Solution. The `LEFT JOIN` keeps all users, with missing order columns filled with nulls for users with no orders. But the `WHERE o.status = 'paid'` then removes every row where the order status is null or not paid, which removes u2 (no orders) entirely and removes u1's refunded row. The result: u1 with 1 and u3 with 1; u2 is missing. The intended result: u1: 1, u2: 0, u3: 1. Fix: move the filter into the join condition (`ON o.user_id = u.id AND o.status = 'paid'`) so that users without paid orders are kept with a count of 0. The lesson: a filter on the right-hand table in `WHERE` turns a left join into an inner join.

Alternative solution. Count users from a separate list and subtract those with paid orders.

Tradeoffs. Dropping zero-order users can overstate an average (averages over only those with orders).

EX-C21-06 · Chain availability

Problem. A feature depends on three services in series with availabilities 99.95%, 99.9% and 99.99%. Compute the chain's availability and the monthly downtime (43,834 minutes in a month). Does it meet a 99.9% objective, and what would you change?

Context. Setting a target.

Data. As above.

Constraints. Six minutes.

Expected approach. FW-C21-06.

Solution. The chain: $0.9995 \times 0.999 \times 0.9999 = 0.9984$, so about 99.84%, which allows $(1 - 0.9984) \times 43,834 =$ about 70 minutes of downtime a month. A 99.9% objective allows 43.8 minutes, so the chain misses it by about 26 minutes. The weakest link (99.9%) contributes most of the downtime. Options: improve that service's reliability, add a second provider or a fallback for it, remove it from the critical path (make it asynchronous or optional), or cache its results so brief outages are masked.

Alternative solution. Lower the promise to 99.8% if the cost of reaching 99.9% exceeds the cost of the extra downtime.

Tradeoffs. Redundancy adds cost and complexity; do the arithmetic of what an outage costs.

EX-C21-07 · On-device, cloud or hybrid for photo search

Problem. A photo-search feature: on-device takes 300 ms; the cloud takes 80 ms compute plus a 100 ms network round trip (180 ms in all) and costs \$0.003 per request. Easy queries are 70% of traffic: accuracy 92% on-device and 93% in the cloud. Hard queries are 30%: 60% on-device and 88% in the cloud. Compute the accuracy of each design, the average latency of a hybrid (easy on the device, hard in the cloud with a 20 ms routing step), and the cloud cost at 500 million requests a day for all-cloud and for hybrid.

Context. A feature design.

Data. As above.

Constraints. Eight minutes.

Expected approach. FW-C21-07 and Worked example 4.

Solution. Accuracy: on-device only $0.7 \times 92\% + 0.3 \times 60\% = 82.4\%$; cloud only $0.7 \times 93\% + 0.3 \times 88\% = 91.5\%$; hybrid $0.7 \times 92\% + 0.3 \times 88\% = 90.8\%$. Hybrid latency: easy 300 ms; hard $20 + 180 = 200$ ms; average $0.7 \times 300 + 0.3 \times 200 = 270$ ms (cloud only is 180 ms; on-device only 300 ms). Cost: all-cloud $500,000,000 \times \$0.003 = \1.5 million a day; hybrid sends 30%: \$0.45 million. The hybrid keeps 90.8% accuracy (0.7 points below cloud only) at 30% of the cloud cost, but is slower than cloud only on average (270 against 180 ms). If latency is the priority, the cloud wins; if cost and privacy matter, the hybrid does. Say which goal you choose.

Alternative solution. Run easy queries on-device and only send a compact summary of hard ones to reduce the privacy exposure.

Tradeoffs. Hybrid adds routing logic and two code paths to maintain.

EX-C21-08 · Read the design document

Problem. A design document proposes to "move user preferences to a new service." List the effort drivers you would look for, ten questions you would ask (use FW-C21-02), and the two you would ask first.

Context. A design review.

Data. None.

Constraints. Seven minutes.

Expected approach. FW-C21-08.

Solution. Effort drivers: a new data model and a migration of existing preferences; the other teams and clients that read preferences (dependencies and old app versions); security and privacy review (preferences may be personal data); scale (reads on every page load); rollout and rollback; ongoing ownership. The two questions to ask first: "What happens to a user's experience while the migration is under way, and how do we roll back if it fails?" and "Who reads these preferences today, including old app versions, and how do they switch?" The rest from the list: riskiest part, request sent twice, freshness, cost at volume, load at ten times, what we are not building.

Alternative solution. Ask for a one-page plan for the migration with a rehearsal date.

Tradeoffs. Too many questions slow the review; pick the few that expose the most risk.

EX-C21-09 · Idempotency arithmetic

Problem. A payments app processes 1,000,000 payments a day of \$50 each. 0.5% of requests are retried because of network failures, and without safeguards 30% of retried requests are processed twice. Compute the duplicate charges per day and their value. How does an idempotency key change this? What does the user see on a retry?

Context. A payments design.

Data. As above.

Constraints. Five minutes.

Expected approach. C21.3.

Solution. Retries: $1,000,000 \times 0.5\% = 5,000$ a day. Duplicates: $5,000 \times 30\% = 1,500$ a day, worth $1,500 \times \$50 = \$75,000$ a day of erroneous charges, plus refunds, support and trust cost. With an idempotency key, the server recognizes a retry of the same request and returns the original result without charging again, so duplicates fall to about zero. On a retry the user sees the original confirmation (not an error or a second charge). Also decide how long keys are remembered (for example, 24 hours).

Alternative solution. Disable the button after the first tap, which reduces user-caused duplicates but does not help network retries.

Tradeoffs. Remembering keys costs storage and needs care for keys reused with different amounts.

EX-C21-10 · A timed full answer

Problem. Give a 20-minute answer to: "Explain how a ride-hailing app shows a driver's car moving on the map in real time, and what could go wrong."

Context. Self-scored; record yourself. State assumptions aloud.

Data. None. Label what you assume.

Constraints. Use the 20-minute budget in section 4.

Expected approach. The technical-fluency answer structure, scored against the six behaviors in section 1.

Solution. An outline of a strong answer. Path: the driver's phone reads its location every few seconds and sends it to the server (battery and data cost, privacy); the server stores the latest location in a fast store, not the main database, since only the most recent point matters; the rider's app needs updates: either it asks every few seconds (polling) or the server pushes updates over a held-open connection (push); push is smoother and cheaper at scale and needs connection management. Smoothing on the client (drawing between points) hides gaps. Constraint: freshness of a few seconds, latency under about a second, and millions of updates per second at scale (throughput). Tradeoffs: update frequency against battery and cost (every 4 seconds against every 1 second is four times the messages); accuracy against privacy (do not show exact location of a rider to the driver before pickup). What can go wrong: poor network (the car seems to jump or freeze, so show a "reconnecting" state and the last known time), stale data (show how old it is), GPS error in tall buildings, the queue backing up in a surge (drop old points, keep the newest), and abuse (limit who can see a driver's location and for how long). Measures: percentile of location age at display, error rate of updates, battery use per trip, support contacts about the map.

Alternative solution. Start from the user's view: what does the rider need to know (arrival time) and design the update rate for that, not for smooth animation.

Tradeoffs. Higher frequency makes the map smoother and costs battery, data and money.

13. Practice protocol

Timed drills (three a week for three weeks).

- *Week 1: the path and plain explanations.* Use IQ-C21-01, IQ-C21-02, EX-C21-01, EX-C21-02 and EX-C21-03. Draw the request path from memory, and explain three terms aloud in 90 seconds each.
- *Week 2: data, caching and reliability.* Use IQ-C21-04, IQ-C21-05, EX-C21-04, EX-C21-05, EX-C21-06 and EX-C21-09. Do the percentile, cache and availability arithmetic by hand, and read a query aloud with the checklist.
- *Week 3: design choices and the full answer.* Use IQ-C21-03, IQ-C21-06, IQ-C21-07, EX-C21-07, EX-C21-08 and EX-C21-10. Practice the on-device versus cloud trade, read a design document with the ten questions, and run a full 20-minute answer with a partner who asks "why is that slow?" midway.

With a partner. Ask your partner to play an engineer who says "it's complicated" and a stakeholder who asks "why can't we just cache it?" Practice asking the focused question and giving the plain explanation.

Sit with an engineer. Ask one engineer on your team to walk you through a request in your product, hop by hop, with real timings, and note one thing that surprised you.

Feedback checklist (yes, partly, no).

1. Did I describe the request path hop by hop and say where time and failure concentrate?
2. Did I use percentiles and a stated target?
3. Did I explain terms plainly and correctly, and say when I was unsure?
4. Did I ask how fresh the data must be before accepting a cache?
5. Did I check what happens if a request is sent twice?
6. Did I check a joined total against a known number?
7. Did I compound dependency availability and tie targets to cost?
8. Did I compare on-device and cloud on more than one factor?
9. Did I ask engineering focused questions, and not tell them how to build?
10. Did I finish with the two-sentence close from section 4?

Answer journal. For each drill record the date, the prompt, the hops or tradeoffs covered, the number you used, your score out of ten and the term you got wrong or avoided.

14. Implementation playbook

Apply the chapter on real work over the next three weeks. Each deliverable doubles as an interview story.

Days 1 to 3. Ask an engineer to walk you through one request in your product (a page load or a core action), with timings. Draw the path (FW-C21-01) and note which hop dominates at the 95th percentile, and what the user sees when each hop fails.

Week 1. Find a number you rely on (a dashboard metric) and trace where it comes from: the event, the pipeline and the transformation, and its freshness. Reconcile it with a second source. Write down any gap.

Week 1. Read one query that produces a number you use, with the checklist in FW-C21-04. Ask the analyst to confirm the grain and the join behavior, and check the total against a trusted figure.

Week 2. Read one design document for a project in your area with the ten questions (FW-C21-02) and pick the two most useful. Ask them in the review and note what you learned.

Week 2. Look up your product's reliability targets (or ask for them): what are they, how much downtime do they allow, what is the error budget status, and what is the dependency chain?

Week 3. For one feature that handles personal data, write a one-page comparison of where it is processed (device, server, partner), what leaves the device and why, and the retention. Review it with privacy or security.

Deliverables that become stories: the request-path map with the dominant hop (a time you understood a system well enough to find the problem), the reconciled number (a time you checked data before trusting it), and the design-document questions (a time you caught a risk early). Keep the numbers real, and keep what you did separate from what the team did.

15. Key takeaways and further reading

Key takeaways

1. Technical fluency means knowing the shape of a system well enough to judge feasibility, cost and risk and to ask good questions; it does not mean writing code, and it includes saying what you do not know.
2. Think in the request path: a user action crosses the device, name lookup, network, server, data store and back; time and failure concentrate in particular hops, and the experience is read at percentiles, not averages.
3. Know the contracts and the data: an API and a schema are hard to change, retries need idempotency, joins can repeat rows, and a number's pipeline and freshness decide what it can tell you.
4. Every architecture pattern is a tradeoff: a cache trades freshness for speed, a queue trades immediacy for resilience, services trade simplicity for independence; ask what each buys, costs and who owns each side.
5. Reliability and privacy are product decisions with numbers: availability compounds across dependencies, an SLO with an error budget turns reliability into a plan, and on-device versus cloud processing trades latency, privacy, capability, cost and offline use, often best resolved by a hybrid with clear disclosure.

Further reading

- Betsy Beyer and others (eds.), *Site Reliability Engineering* (2016), on service level objectives, error budgets and incident practice [V: [Site Reliability Engineering](#), O'Reilly, 2016].
- Martin Kleppmann, *Designing Data-Intensive Applications* (2017), a thorough reference for databases, caching, queues and consistency [V: [O'Reilly](#), March 2017, ISBN 9781449373320].
- Jakob Nielsen, "Response Times: The 3 Important Limits," Nielsen Norman Group, the source of the 0.1, 1 and 10 second thresholds (from his 1993 book *Usability Engineering*, building on Miller, 1968) [V: [Nielsen, Usability Engineering](#) (1993), chapter 5, republished by Nielsen Norman Group; thresholds trace to Miller (1968)].

- Joel Spolsky, "The Law of Leaky Abstractions," Joel on Software, 11 November 2002 [V: Joel Spolsky, Joel on Software, 11 November 2002].
- Peter Deutsch and others, the "fallacies of distributed computing," as discussed in engineering references [V: Fallacies of distributed computing: first four from Bill Joy and Tom Lyon, three more from L. Peter Deutsch (1994), the eighth from James Gosling (about 1997), all at Sun Microsystems].
- Gene Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," *AFIPS Conference Proceedings* 30 (1967), pp. 483 to 485, and any standard summary of Amdahl's law [V: AFIPS Conference Proceedings 30, Spring Joint Computer Conference, 18 to 20 April 1967, pp. 483 to 485 (IEEE reprint)].
- Meta, "More details about the October 4 outage" (5 October 2021; engineering.fb.com/2021/10/05/networking-traffic/outage-details/), and Cloudflare's analysis of the same event (blog.cloudflare.com/october-2021-facebook-outage). The sources for CS-C21-02.
- Your own company's published API documentation and status page, as examples of contracts and of how incidents are explained.